LLM based automatic code generation for Web applications and Restful APIs

Project Team

Muhammad Ubaid Ul Rehman	21I-1749
Ahsan Nadeem	21I-1710
Kanwar Muhammad Daniyal	21I-1180

Session 2021-2025, **F24-084-R-WebCodeGen**

Supervised by

Mr. Irfan Ullah

Department of Data Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2025

Contents

1 Introduction **1**

1.1 Problem Domain 1

1.2 Research Problem Statement 2

2 Literature Review **3**

2.1 Related Research 3

2.1.1 Research Item 1 3

2.1.1.1 Summary of the research item 3

2.1.1.2 Critical analysis of the research item 4

2.1.1.3 Relationship to the proposed research work 4

2.1.2 Research Item 2 5

2.1.2.1 Summary of the research item 5

2.1.2.2 Critical analysis of the research item 5

2.1.2.3 Relationship to the proposed research work 6

2.1.3 Research Item 3 6

2.1.3.1 Summary of the research item 7

2.1.3.2 Critical analysis of the research item 7

2.1.3.3 Relationship to the proposed research work 8

2.1.4 Research Item 4 8

2.1.4.1 Summary of the research item 8

2.1.4.2 Critical analysis of the research item 9

2.1.4.3 Relationship to the proposed research work 9

2.1.5 Research Item 5 10

2.1.5.1 Summary of the research item 10

2.1.5.2 Critical analysis of the research item 10

2.1.5.3 Relationship to the proposed research work 11

2.1.6 Research Item 6 12

2.1.6.1 Summary of the research item 12

2.1.6.2 Critical analysis of the research item 12

2.1.6.3 Relationship to the proposed research work 13

2.2 Analysis Summary of research items 13

3 Proposed Approach **21**

References **23**

Chapter 1

Introduction

Code generation has revolutionized software development by automating repetitive tasks, improving code quality, and reducing development time. Template-based code generation is an efficient technique among these because of its flexibility and ease of use, especially in web and API development.

However, existing methods require manual intervention, which limits their flexibility and efficiency. Our research focuses on addressing these challenges by automating template and code generation. This approach allows for modification of templates, which as result create CRUD operations, backend interfaces, routing, ORMs, and web APIs automatically. For the research work, we use synthetically generated XML and template datasets as our case study.

1.1 Problem Domain

In traditional software development, the creation of foundational structures such as code templates, boilerplate code, and business logic is a labor-intensive process, along with the probability of errors due to manual efforts. In today's era, our software systems are continuously increasing in complexity, maintaining consistency across projects to best practices becomes challenging. In this competitive and modern market manual efforts to generate these templates not only slow down the development process but also introduce potential errors that can compromise the quality and scalability of the final product.

Current methods of automatic code generation still require significant developer's input, lack flexibility, and do not fully eliminate the risk of human error. The growing need for scalable, maintainable, and error-free code generation in domains like web applications, APIs, and database interactions, highlights the limitations of these manual and semi-automated approaches.

In our case study, we intend to deal with the problem of the inefficiency lies in manually generating and maintaining templates and code structures, which leads to delays and inconsistencies. Therefore, the problem domain revolves around the need for a system that automates template generation, reduces manual intervention, and produces consistent, error-free code aligned with best practices across multiple projects and environments.

1.2 Research Problem Statement

Current approaches for automatic code generation usually based on prompt based code generation and requires manually copy pasting the generated code into the files, which consequently requires manual effort at some point. State of the art, template based code generation approaches deal with manual, hard-coded and ready-made templates only. So far, there does not exist approaches that automatically generates templates based on user requirements provided via class diagram. In this research work, we intend to develop an approach to automate the whole generation tasks which includes not only the code generation but also the templates generation.

Our research aims to address the gap by investigating an automated template generation approach that utilize class diagrams to produce template code and business logic. Our study seeks to determine how accurately the automation can improve consistency, reduce human error, and enhance productivity, ultimately providing more scalable and reliable automated software development practices.

Introducing automated template based code generation addresses the inefficiencies and inconsistencies which comes with the manual generation of templates for web applications and APIs. In traditional software development process, these templates are manually created which require time, effort and additional man force which delays the development process that impacts on the quality of the final product. Additionally, when application's complexity increases, the manual approach is not feasible. Our research aims to solve these problems by automating the template generation through class diagrams. The software ensures will provide template code and associated files also ensuring consistent and error free industry standard code. This surely will accelerate the development process while reducing the likely human errors. Ultimately, enhancing the productivity for developers, by providing a tool which allows them to focus on higher level tasks rather than repetitive tasks in coding.

Chapter 2

Literature Review

This chapter critically examines existing literature relevant to the research topic. It identifies key studies, theories, and findings, providing a foundation for the proposed research and highlighting its relationship to prior work.

2.1 Related Research

2.1.1 Research Item 1

Template-based automatic code generation for web application and APIs using class diagram. [4]

2.1.1.1 Summary of the research item

The research in this paper focuses on automating the tedious, repetitive tasks involved in web application and API development by using class diagrams to generate CRUD-based business logic, ORM, backend UIs, and API routes. The approach leverages Python and blackboard architecture, where different knowledge sources use templates to map class attributes to various code structures. This method offers language and pattern independence, allowing for flexibility in template modification and import. The key innovation is the automation of manual development tasks that previously required significant developer input, making the process both faster and more efficient.

The authors conducted empirical studies to evaluate the performance of their approach, particularly comparing it to traditional manual development processes. The results showed a significant reduction in development time—manual tasks that typically took hours were completed in less than a minute using the code generator. Three experiments were conducted: manual coding without templates, manual coding with templates, and automated

code generation, with the latter proving the most efficient. This research highlights the potential for reducing labor-intensive coding while maintaining flexibility and scalability across different platforms.

2.1.1.2 Critical analysis of the research item

Strengths:

The authors' method offers significant advantages in terms of efficiency. Their system automates repetitive coding tasks, significantly reducing development time. In an experiment, tasks that typically took hours were automated and completed in seconds, demonstrating substantial time savings. This approach is language and framework independent, working across various programming environments. The ability to import, modify, and reuse templates makes it customizable and adaptable to different projects. The automation of tasks like CRUD operations and API routes minimizes human error and ensures consistent coding standards. This research provides broad applicability across various use cases, making it a versatile solution capable of handling different layers of development.

Weaknesses:

While the system effectively reduces development time, a more comprehensive evaluation is needed to assess its overall value.

Factors like code quality, performance, scalability, and maintainability should be considered alongside development speed. Additionally, the system's focus on backend development neglects the crucial role of frontend code generation. While it handles backend logic and APIs well, its ability to generate complex frontend components is limited.

Scalability and adaptability are also concerns. Although tested on smaller projects, the system's performance with larger, more complex systems remains uncertain. The generator's handling of intricate class diagrams and domain models could be further clarified. Moreover, the system's reliance on templates, while beneficial, can become cumbersome in large projects with frequent customization needs. Finally, the system's dependence on accurate class diagrams can be a potential pitfall. Inconsistent or outdated diagrams could lead to errors in the generated code. Moreover, there is no method discussed to check whether the generated code performs the desired functionality.

2.1.1.3 Relationship to the proposed research work

This paper serves as the base paper for our research. The approach we are proposing improves the method laid down in this paper. The manual templates approach is not scale able, for this approach we are proposing automatic template generation through class diagrams. In addition to that validation of code is being introduced, Both of the proposed

methods will ensure correctness of code and will further decrease the development time.

2.1.2 Research Item 2

Code Generation from UML State Chart Diagrams.[1]

2.1.2.1 Summary of the research item

This research paper presents a method to generate code from UML state charts in event-driven systems. As the complexity of these state diagrams increases, it becomes challenging to automatically generate code efficiently, especially without increasing code coupling. The paper proposes a novel design pattern to address this issue, incorporating hierarchical, concurrent, and history states. This design pattern extends the existing state pattern by introducing object composition and delegation mechanisms. It also introduces mapping rules to facilitate code generation by defining transformation rules from state chart elements to Java code. The transformation is implemented using Acceleo, a template-based code generation tool, ensuring a structured and efficient generation process. The effectiveness of the proposed method is validated through a case study of a microwave oven control system.

The new design pattern optimizes code generation by reducing complexity through features like deep history and concurrency, which manage multiple domains of composite states. The transformation rules ensure that the generated code is more modular, reusable, and maintainable compared to traditional methods like JCode and Template HHCStateMachine. A comparison of these methods shows that the new pattern proposed in this paper generates fewer lines of code and reduces generation time, making it more efficient, especially for complex systems. The paper concludes by outlining future work, which will involve building a general transformation framework and enhancing testing modules.

2.1.2.2 Critical analysis of the research item

Strengths:

The research introduces a novel design pattern that extends the traditional state pattern by incorporating object composition and delegation, this pattern is particularly effective for managing complex state hierarchies, concurrency, and history states in UML diagrams.

The paper provides a comprehensive framework for code generation by defining clear mapping rules from UML state charts to Java code. Acceleo, a template-based generation tool, enhances the practicality of the solution.

A significant benefit of this approach is its ability to reduce code redundancy and improve maintainability. Compared to other methods, this leads to more modular and loosely coupled code, contributing to long-term software sustainability.

The research validates the method through a case study involving a microwave oven control system. This demonstrates its applicability to real-world scenarios with concurrent and deep history states.

Additionally, the paper offers a comparative analysis with existing methods. It highlights improvements in code generation efficiency and time savings, supported by quantitative data, strengthening the overall argument.

Weaknesses:

While the research validates the method through a specific case study, its applicability is limited. It doesn't explore broader domains like distributed systems or real-time systems, hindering the generalizability of its findings.

The framework's focus on Java code restricts its use to Java-centric environments. A more versatile, language-agnostic approach would expand its applicability. Additionally, the reliance on Acceleo for template-based generation can be a limitation. Acceleo may have a steep learning curve or integration challenges in different development environments.

While the research demonstrates efficiency in code generation, it lacks performance metrics for the generated code. Execution speed, memory usage, and scalability under real-world conditions are not evaluated.

Finally, the research offers minimal discussion on testing and debugging. Although unit testing is mentioned as future work, a stronger focus on these areas would improve the reliability of the approach.

2.1.2.3 Relationship to the proposed research work

The above research paper closely relates to our research because in our method the code generation from Class Diagram is a fundamental part. In this paper they are generating code from UML State chart diagrams which helps us to understand how can code can be generated from a diagram. Furthermore, this is a Java anguage specific system. In response to this we are proposing code generation in multiple languages. The testing and debugging of code is discussed but limited to unit testing which can guide us in our research, how can we validate our generated code.

2.1.3 Research Item 3

Template-based Code Generator for Web Applications.[5]

2.1.3.1 Summary of the research item

The paper discusses a template-based code generator developed to streamline the creation of web applications. It highlights the growing complexity and size of modern web applications and presents code generators as an efficient solution for simplifying development, reducing errors, and enhancing productivity. The code generator described uses templates to automate the generation of code for various web application components, including front-end and back-end modules. The generator was integrated into a real-life ERP system, proving its viability and achieving significant reductions in development time (98.95%), test runs (93.97%), and code size (49.37%) compared to manual coding.

The system leverages technologies such as stored procedures, HTML templates, and JSON structures, enabling it to generate structured, reusable, and bug-free code. A key aspect of the generator's efficiency is the use of validated reference implementations to create templates, which ensures that the generated code is optimized and error-free. The paper also emphasizes the positive impact on developer productivity and resource utilization, although it acknowledges the potential inflexibility when needing different user interface types. The success of the tool demonstrates the advantages of using template-based code generation in industrial web applications.

2.1.3.2 Critical analysis of the research item

Strengths:

The research addresses the growing complexity of modern web applications by automating code generation. This approach aims to reduce the time-consuming and error-prone nature of manual coding.

A thorough empirical evaluation highlights the benefits of the code generator, including significant reductions in development time and bug count. The study used real development tasks from a large ERP system to enhance external validity.

The code generator was successfully integrated into an industrial environment, demonstrating its feasibility and scalability. The system leverages modern web technologies, making it flexible and easy to adopt.

By automating code generation, the system significantly reduces human error and improves developer productivity. This leads to fewer bugs, reduced development time, and minimized test runs, ultimately enhancing project efficiency.

Weaknesses:

The code generator boasts efficiency and consistency, but it's not without limitations. Developers face restricted UI customization due to pre-defined templates. Additionally, the system's effectiveness relies heavily on well-designed templates, as poorly designed

ones can hinder future modifications. Concerns about bias arise from a single developer's involvement in both development and evaluation. A wider range of web applications and developers in the evaluation process would provide a more comprehensive picture.

Furthermore, the paper doesn't address potential security vulnerabilities or performance impacts of large-scale automatic code generation. Finally, the system's dependence on specific technologies limits its use for organizations with different tech stacks, requiring significant modifications for broader applicability. To fully understand the code generator's value proposition, a comparison with existing solutions is needed.

2.1.3.3 Relationship to the proposed research work

This paper proposes a method which generates code for web applications through manual templates, which as we discussed before we are automating to reduce time. It also lays down the advantages of using a template based approach which helps us to further solidify our research point of view.

Moreover, the technologies used in this paper which ensure the structure, re-usability and bug free code is an important point which helps us in our proposed way of validating the generated code.

2.1.4 Research Item 4

Template-Based Code Generation Framework for Data-Driven Software Development[3]:

2.1.4.1 Summary of the research item

This research paper presents a framework aimed at addressing the growing need for efficient and rapid software development, especially for database-oriented applications. The primary objective is to automate the generation of production-ready code using customizable templates, thereby reducing development time and cost. This framework allows developers to focus on business logic rather than repetitive coding tasks. It is particularly useful for web services, mobile computing, and IoT applications, which often require a structured database layer. The framework supports code reuse, maintains uniform code quality, and provides productivity gains by generating consistent and well-structured code across different layers (data access, business logic, and stored procedures). The system uses Visual Studio and .NET technologies to create a library of templates that can be customized for various applications. The framework also addresses common software development challenges such as minimizing software defects, managing a wide range of

technologies, and improving security and performance. By generating code automatically, the tool enforces best practices and reduces the time required for debugging and testing. The paper concludes with a discussion of the potential future work in expanding the front-end interaction and enhancing the framework's versatility.

2.1.4.2 Critical analysis of the research item

Strength:

The framework provides substantial productivity gains by automating the generation of repetitive code, allowing developers to focus on higher-level logic and business needs. This automation not only accelerates development but also ensures uniform code quality, as it enforces a consistent coding style and structure across all application components, reducing the likelihood of errors and improving maintainability. Furthermore, the templates used in the framework are highly reusable across multiple projects, which shortens development cycles for future applications and makes software maintenance more efficient. The framework is also highly flexible, supporting integration with various platforms and technologies, including SQL, C#, .NET, Java, and others, enhancing its applicability across diverse projects. Additionally, the tool strengthens security by enforcing best practices, while performance is optimized through measures that reduce vulnerabilities like SQL injection and improve database query efficiency.

Weaknesses:

While the framework excels at generating back-end code, it has limited support for creating fully functional front-end applications, which restricts its applicability across the entire development stack. Additionally, although the templates are customizable, creating and adjusting them requires a deep understanding of the underlying technologies, potentially adding complexity for less experienced developers. The framework also heavily relies on .NET and Visual Studio, limiting its use to developers within these ecosystems and reducing its accessibility for those working with other technologies. Moreover, despite addressing security, the framework still requires developers to properly configure and implement security measures, leaving room for human error and potential vulnerabilities in the generated code.

2.1.4.3 Relationship to the proposed research work

Both approaches focus on automating the code generation process to enhance development efficiency and reduce manual effort, particularly in generating repetitive code such as CRUD operations and backend infrastructure. They emphasize improving code quality and consistency through the use of template-based systems, allowing developers to focus on more complex tasks. Automation in both cases leads to increased productivity, as

well as ensuring that generated code adheres to best practices. One approach targets web applications and APIs, using class diagrams to automate the creation of backend components, while the other is centered on data-driven software development, optimizing for security, performance, and error reduction. Both methods share the goal of streamlining the development process by providing scalable and flexible automation solutions across various development contexts.

2.1.5 Research Item 5

A Comparative Review of AI Techniques for Automated Code Generation in Software Development: Advancements, Challenges, and Future Directions[2]

2.1.5.1 Summary of the research item

Above paper provides an extensive examination of how Artificial Intelligence (AI) is transforming the software development lifecycle, particularly focusing on Automated Code Generation (ACG). It compares various AI techniques such as Rule-Based Systems, Machine Learning (ML), Natural Language Processing (NLP), Deep Learning (DL), and Evolutionary Algorithms (EAs). These techniques have enabled developers to automate repetitive coding tasks, allowing them to focus more on design and problem-solving. The paper highlights the benefits AI brings to ACG, including increased productivity, improved code quality, and support for code generation from alternative representations such as natural language descriptions and sketches. However, it also addresses the challenges related to data availability, scalability, performance, and ethical concerns such as privacy, security, and potential bias in AI-generated code. The paper concludes by discussing future directions and research opportunities, aiming to improve the reliability, scalability, and flexibility of AI-driven ACG systems.

2.1.5.2 Critical analysis of the research item

Strength:

The paper offers a comprehensive comparison of various AI techniques for automated code generation, providing valuable insights into the trade-offs between different approaches, which is highly beneficial for researchers and developers. It effectively highlights the key benefits AI brings to software development, such as increased productivity, improved code quality, and the capability to generate code from diverse input forms, including natural language descriptions. Additionally, the paper addresses critical ethical considerations associated with AI in code generation, including bias, transparency, and privacy, making it particularly relevant to ongoing discussions surrounding AI ethics.

Furthermore, through case studies and real-world applications, the paper demonstrates the practical implications of AI, showing how it can enhance coding efficiency and significantly reduce development times.

Weaknesses:

The paper, while providing valuable insights into various AI techniques for automated code generation, falls short in offering detailed guidance on implementation. Although it discusses the benefits of AI in software development, it lacks concrete examples or step-by-step instructions on how to integrate these techniques into existing development workflows. This absence of practical implementation details leaves a gap for developers who may need more direction on applying these advanced AI methods in real-world projects.

Another limitation highlighted in the paper is the challenge of scalability. While the paper acknowledges that scalability is an issue, particularly in large-scale applications, it does not provide sufficient solutions or strategies for overcoming these challenges. Developers and organizations working with extensive codebases may struggle to apply these AI techniques effectively due to the lack of guidance on how to scale the solutions.

The AI techniques discussed in the paper also come with heavy data requirements, which can be a significant barrier to their application. Many of these methods require vast amounts of high-quality training data to be effective. Obtaining such data can be particularly difficult in niche or less-explored domains, limiting the accessibility and practicality of these techniques for some developers and organizations.

Additionally, the paper notes that the AI techniques often struggle with understanding the context and intent behind code generation tasks. This limitation becomes particularly apparent in more complex or domain-specific scenarios, where a deep understanding of the underlying logic and requirements is critical. As a result, these AI techniques may produce suboptimal code when faced with ambiguous or intricate coding tasks, limiting their overall effectiveness in certain applications.

2.1.5.3 Relationship to the proposed research work

This work is particularly relevant because it directly tackles the common bottlenecks in software development, such as repetitive and time-consuming coding tasks. By automating the generation of backend infrastructure, CRUD operations, and web APIs, it significantly reduces the manual effort required in these areas. This not only speeds up development but also minimizes the chances of human error, ensuring that the code produced is consistent and adheres to quality standards. The use of class diagrams as input for code generation also allows for a more structured and organized development process, which helps maintain clarity and reduce complexities in project management. Ultimately,

the approach enhances productivity by allowing developers to focus on more complex, high-level tasks, making the development process more streamlined and efficient. This is particularly valuable in fast-paced environments where quick iteration and deployment are crucial.

2.1.6 Research Item 6

Projective Template-Based Code Generation[6]

2.1.6.1 Summary of the research item

This paper introduces a novel approach to template-based code generation (TBCG) that separates concerns of the model, template, and control code. Traditional TBCG methods often suffer from mixing control logic and textual patterns, which complicates readability, reusability, and flexibility. The authors propose "projective templates," inspired by relational theory, where templates act as annotated relations with no explicit control code. This approach eliminates the need for imperative control constructs like loops or conditionals, making the templates cleaner and more abstract.

The system uses relational operators to retrieve, manipulate, and substitute data into the templates, allowing for more efficient and scalable code generation. The paper also provides a use case to demonstrate the applicability of the system in generating SQL code for comparing table contents across different databases. The authors argue that projective templates are more readable, concise, and flexible, leading to enhance code generation scenarios that can handle complex text patterns.

2.1.6.2 Critical analysis of the research item

Strengths

The approach effectively separates the model, template, and control logic, enhancing the readability, maintainability, and reusability of the code templates. By eliminating the need for control code within the templates, it provides a simpler and more abstract representation of the target code, improving clarity and reducing complexity. The control logic, inferred from the relational structure of the templates, is inherently reusable across different templates and scenarios, promoting consistency and minimizing redundancy. The projective template system is scalable, capable of handling complex code generation scenarios without compromising performance or clarity, making it suitable for larger and more intricate applications. Additionally, its use of relational operators provides flexibility in generating diverse types of code, from SQL to object-oriented programming constructs, and allows it to accommodate different text formats and data structures.

Weaknesses:

The system's lack of support for imperative control structures, such as loops and conditionals, limits its flexibility in scenarios requiring more complex data transformations. While this limitation is intentional, it may hinder use cases that involve dynamic model manipulation. Additionally, the powerful generation capabilities could lead to over-reliance on automated code generation, resulting in excessive or unnecessary code that might be simpler if written manually. Changes in the model or template can also have significant ripple effects on the generated code, potentially creating maintenance challenges if not carefully managed. Moreover, the system introduces unique concepts, such as annotated projection operators, which may require a learning curve for developers unfamiliar with the relational approach, limiting its immediate accessibility.

Finally, the approach's reliance on relational models can be restrictive for non-relational data or less structured generation tasks, potentially limiting its overall scope of application.

2.1.6.3 Relationship to the proposed research work

Both works focus on improving software development efficiency through the automation of code generation, reducing repetitive tasks such as CRUD operations, business logic, and backend infrastructure creation. They use template-based systems to generate structured, consistent, and reusable code, minimizing errors and enhancing maintainability. By automating these processes, they significantly decrease development time and ensure that the generated code follows industry best practices, resulting in better performance, security, and faster production-ready outputs. This streamlines the development lifecycle and allows developers to concentrate on more complex tasks, boosting productivity across projects.

2.2 Analysis Summary of research items

Title	Summary	Strengths	Weaknesses
Template-based Code Generator for Web Applications[5]	• Template-based code generator simplifies web app development. • Reduces complexity, errors, and enhances productivity. • Automates front-end and back-end code generation. • Integrated into a real-life ERP system. • Utilizes stored procedures, HTML templates, and JSON for structured, reusable, bug-free code. • Uses validated reference implementations to optimize code quality. • Improves developer productivity and resource use.	• Evaluation metrics chosen for validity and reliability of measures. • Experiments conducted to compare automatic code generation with manual implementation.	• Code generator inflexibility in user interface creation beyond HTML templates. • Development workload similar to manual implementation when adding templates.

Title	Summary	Strengths	Weaknesses
Projective Template-Based Code Generation[6]	• Projective templates enhance code generation with relational projective templates. • Templates separate textual patterns and control code for readability and clarity. • Conditional generation is possible without explicit control code.	• Separates model, template, and control logic for better readability, maintainability, and reusability. • Eliminates control code, simplifying the representation of target code and reducing complexity. • Control logic is reusable across scenarios, ensuring consistency and minimizing redundancy. • Scalable and flexible for generating diverse code types, accommodating different formats and structures.	• Lacks support for imperative control structures, limiting flexibility in complex data transformations. • Over-reliance on automated code generation could result in excessive or unnecessary code. • Changes in the model or template may create significant maintenance challenges. • Relies on relational models, which may restrict use with non-relational or less structured data. • Transformation should not change the source model.

Title	Summary	Strengths	Weaknesses
Code Generationfrom UML State Chart Diagrams.[1]	• Proposes code generation from UML state charts for event-driven systems. • Addresses complexity and code coupling with a novel design pattern. • Pattern includes hierarchical, concurrent, and history states using object composition and delegation. • Introduces mapping rules for transforming state chart elements to Java code via Acceleo. • Case study on microwave oven system validates the method's effectiveness. • Optimizes code generation with modular, reusable, and maintainable code.	• Novel Design Pattern. • Comprehensive Solution for Code Generation. • Efficiency and Reusability. • Practical Validation. • Clear Comparative Analysis.	• Lack of Flexibility in UI Generation. • Limited Scope of Application. • Dependence on Java. • Tool Dependency. • Lack of Performance Metrics on Generated Code. • Minimal Discussion on Testing and Debugging.

Title	Summary	Strengths	Weaknesses
Template-Based Code Generation Framework for Data-Driven Software Development[3]	• Template-based code generator for data-driven software development. • Enhances security, reliability, and efficiency in software development processes. • Improves workflow by automating project setup tasks. • Focuses on generating runnable files for easy integration and customization.	• Automates repetitive code generation, boosting productivity and enabling focus on higher-level logic. • Ensures uniform code quality and structure, reducing errors and improving maintainability. • Templates are reusable across projects, shortening development cycles and easing maintenance.	• Limited support for front-end applications, restricting full-stack applicability. • Customizable templates add complexity, especially for less experienced developers. • Heavily Heavily relies on .NET and Visual Studio, reducing accessibility for other technologies.

Title	Summary	Strengths	Weaknesses
Template based Automatic code generation for Web application and APIs Using Class Diagram.[4]	• Automates repetitive web app and API development tasks. • Uses class diagrams to generate CRUD logic, ORM, UIs, and API routes. • Python-based with blackboard architecture and flexible templates. • Reduces manual coding time significantly. • Empirical tests show automated generation is faster than manual methods. • Empirical tests show automated generation is faster than manual methods.	• Efficiency in Development Time. • Language and Framework Independence. • Template Flexibility. • Automation of Tedious Tasks. • Broad Applicability Across Use Cases:.	• Narrow Evaluation Metrics. • Lack of Focus on Frontend Code Generation. • Limited Scalability Evaluation. • Potential for Template Overhead. • Dependence on Accurate Class Diagrams.

Title	Summary	Strengths	Weaknesses
A Comparative Review of AI Techniques for Automated Code Generation in Software Development: Advancements, Challenges, and Future Directions[2]	• Examines AI techniques (e.g., ML, NLP, DL, EAs) for Automated Code Generation (ACG). • Highlights AI benefits like increased productivity, improved code quality, and alternative code generation methods. Addresses challenges like data availability, scalability, performance, and ethical concerns in AI-generated code.	• Provides a comprehensive comparison of AI techniques for automated code generation, highlighting trade-offs. • Emphasizes AI benefits like increased productivity, improved code quality, and diverse input methods. • Addresses ethical considerations such as bias, transparency, and privacy in AI-generated code.	• Lacks detailed implementation guidance for integrating AI techniques into development workflows. • Scalability challenges are acknowledged but not addressed with sufficient solutions. • Heavy data requirements make AI techniques less accessible, especially in niche domains. • AI struggles with understanding context and intent, leading to suboptimal code in complex scenarios.

Chapter 3

Proposed Approach

In this research, we have proposed a modular approach and aims to develop an automated tool that generates templates for web applications and APIs using class diagrams as the foundational input. The tool will parse class diagrams to extract the necessary components, such as classes, attributes, methods, and relationships, and then use this information to create standardized, customizable templates.

These templates will include the essential boilerplate code required for web applications (e.g., models, controllers, views) and APIs (e.g., endpoint definitions, data models).

Methods:

Class Diagram Drawing Interface: A facility to draw the UML diagrams, mainly class diagrams, within our application. This feature will free the user from cross-platform usage. Our drawing feature of application will provide all the elements such as, relationships, classes, functions and attributes need for any UML diagram. Once the diagram is ready user can upload it directly from there on a single click. Which be later on use for code generation.

Class Diagram Parsing: This is the first step after drawing for code generation. The provided diagram will be parsed into XML format. As we will be using LLM models for our code and template generation, the best way we can utilize them is to provide data that is easy to understand and better to train the model, XML format will serve the purpose. Once the parsing is done, we will provide the parsed data to the model for further processing.

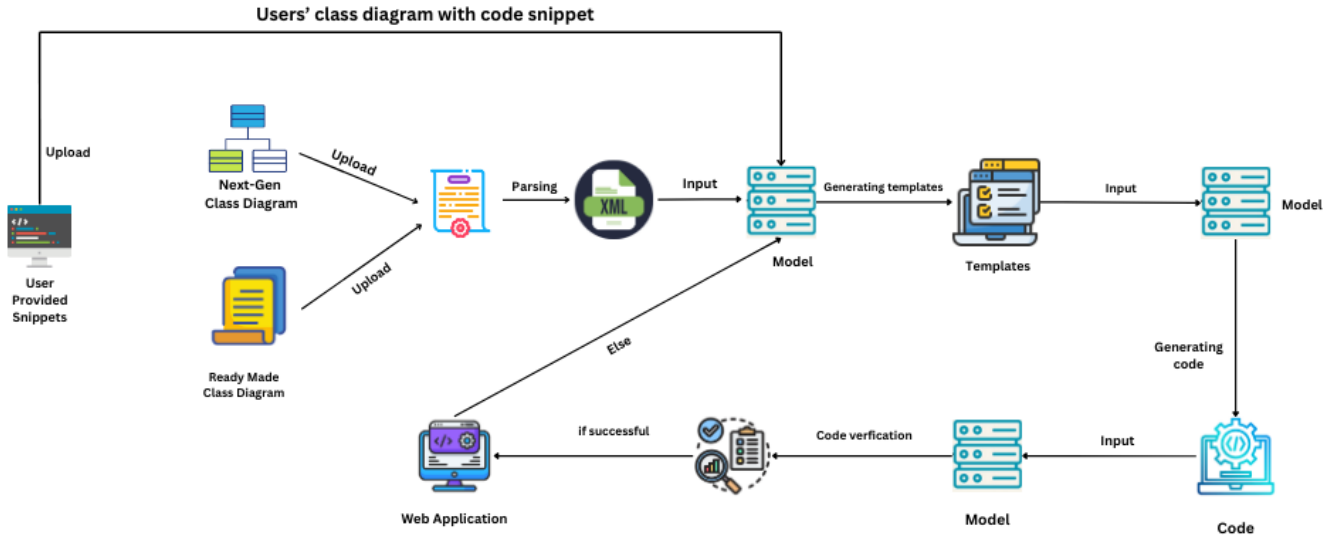
Template Generation Module: Based on the XML input of the class diagram, the template generation model will generate the foundation code for the web application and API. It will be generating model functions, classes, controller actions, views, and API endpoints; depending on the type of application that is being requested via class diagram. Additionally, user can also provide its own templates rather then generating from scratch for its application.

Code Generation Module: When the initial templates are generated, the code generation module will be responsible for generating actual runnable code and main business logic based on the user requirements.

Testing and Validation: Our code validation model will test and validate the generated code for both templates and business logic. A built-in testing and validation suite will automatically check the generated templates for consistency, correctness, and adherence to coding standards. This ensures that the generated code is ready for immediate use in the development process.

Each iteration will focus on enhancing the accuracy of the parsing process, the flexibility of the generated templates, and the usability of the customization module. By employing these methods, our research aims to significantly reduce the time and effort required to generate consistent and high quality templates for web applications and APIs, ultimately improving the overall efficiency of the software development process.

To evaluate our project, we consider different class diagrams and ready made templates for code generation s as a case study. Our aim in this case study is to automate the whole generation tasks from templates to end product. The dataset used for this research work is synthetically generated XML and template codes, and for technical evaluation we will use count of missing classes and functions for the generated code , and F1 score and accuracy for our generative models.



Bibliography

[1] Xingchan Li and Xinquan Wu. Code generation from uml state chart diagrams. In *2022 International Conference on Management Engineering, Software Engineering and Service Sciences (ICMSS)*, 2022.

[2] Nada Mohammed Abdul Odeh, Ayman Odeh. A comparative review of ai techniques for automated code generation in software development: Advancements, challenges, and future directions. *TEM Journal. 726-739. 10.18421/TEM131-76.*, pages 726–739, 02 2024.

[3] Kshitija Shinde and Yu Sun. Template-based code generation framework for data-driven software development. *2016 4th Intl Conf on Applied Computing and Information Technology/3rd Intl Conf on Computational Science/Intelligence and Applied Informatics/1st Intl Conf on Big Data, Cloud Computing, Data Science Engineering (ACIT-CSII-BCD)*, 2016.

[4] Irfan Ullah and Irum Inayat. Template-based automatic code generation for web application and apis using class diagram. In *2022 International Conference on Frontiers of Information Technology (FIT)*, pages 332–337, 2022.

[5] BURAK UYANIK and VEYSEL HARUN (2020) ŞAHİN. A template-based code generator for web applications,. *" Turkish Journal of Electrical Engineering and Computer Sciences: Vol. 28: No. 3, Article 37.*, 2020.

[6] Chared Zeev and Tyszberowicz Shmuel. Projective template-based code generation. *TEM Journal*, pages 726–739, 02 2024.